UNIVERSITY OF SALFORD
School of Computing Science and Engineering

Reinforcement Learning for Adaptive Control Systems: Temperature Control Systems in an HVAC Unit

Submitted in partial fulfillment of the requirements for the degree of:
MSc Advanced Control Systems
August 2024

Jesutomito Morakinyo

Supervisor: Dr. Martin Burby

University of Salford

School of Computing Science and Engineering

Student Name: Jesutomito Morakinyo

Course Code: Dissertation

Project Title: Reinforcement Learning for Adaptive Control Systems: Temperature Control Systems in a HVAC Unit

I certify that this report is my own work. I have properly acknowledged all material that has been used from other sources, references, etc.

Student Signature: ..... **Date:** .....

Official stamp:

Submission date (to be entered by relevant School Office staff): .....

ABSTRACT

This dissertation focuses on the exploration of the applications of reinforcement learning (RL) for adaptive control of heating, ventilation, and air conditioning (HVAC) systems. In this dissertation, there is a major focus on the use of strategies like the exploration-exploitation trade-offs to optimize the performance of the RL model. A Q-learning reinforcement learning model will be developed with the use of Python and Jupyter Notebooks, to maintain the optimal temperature settings within HVAC systems. A simulated environment will be created to model a room's temperature dynamics, factors that will be considered include room size, external temperature, insulation, and HVAC power. There will also be a comparison of the performance of the reinforcement learning-based adaptive control models with traditional PID control approaches with accuracy, energy consumption, and response time being the basis of comparison between these various adaptive control models. Different exploitation-exploration strategies will be used to achieve energy efficiency and temperature control and as a result, there will be an evaluation to determine how effective these strategies have been. After all these, there will be proposals for enhancements in the works of reinforcement learning algorithms to give room for further improvements in this area in terms of adaptability and robustness.

Keywords – Heating, Ventilation, And Air Conditioning (HVAC), Q-Learning, Python, PID Control, Exploration-Exploitation Trade-Offs.

ACKNOWLEDGEMENTS

First and foremost, I would like to express my deepest gratitude to my supervisor, Dr Martin Burby for his support, expertise and guidance throughout this incredible journey. His encouragement and advice have brought me a long way, and I appreciate it.

I would also like to thank my lecturers, Prof. TX Mei, Prof. Mary He, Dr Saleem Ameen, and Dr Gwowu Wei for pushing me to achieve high standards in my work.

I especially appreciate my coursemates at the University of Salford, whose camaraderie, collaborations and discussions have elevated my research experience.

Finally, I would like to thank my family and friends for their endless support and encouragement. To my parents, Pastor and Pastor (Mrs.) Morakinyo, thank you for instilling the invaluable vesture of hard work and dedication in me, it has helped me a lot on this journey. To my partner, Esranur Tat, thank you for your patience, understanding, and for always being there when challenges arose. I sincerely appreciate everyone who has contributed to this project in any way or form, whether big or small, your efforts have not been in vain.

TABLE OF CONTENTS

LIST OF FIGURES

1. INTRODUCTION

Heating, ventilation, and air conditioning (HVAC) systems are very important for making sure that there is indoor comfort and that the quality of air in various building types like commercial, residential, and industrial structures is kept high. This can be achieved by the regulation of air circulation, temperature, and humidity to make sure that the occupants of that building have a high quality of life and that the building operates efficiently. It is important to note that HVAC systems are responsible for very high energy usage. statistically, they are responsible for close to 40% of the total energy use in an average building. There is a globally growing concern about the scarcity of energy, and an urgent need to reduce carbon emissions, therefore, a successful and effective approach towards optimizing the efficiency and the performance of HVAC systems has also become one of the most pressing issues worldwide.

There are traditional HVAC control methods, and they mostly depend on PID (Proportional-Integral-Derivative) controllers. The reliability and simplicity of PID controllers make them widely used, but they do fall short when it comes to their dynamic and their performance in an unknown environment. In the application of PID controllers, predefined rules are set, and they require precise tuning. This makes them struggle with real-time changes in an environment and in cases where the occupancy might fluctuate. In a bid to dynamically improve the adaptiveness to the varying conditions and to also improve the efficiency of HVAC systems, interest in adopting different advanced control strategies has increased a lot.

In machine learning (ML), reinforcement learning (RL) which is a subset of ML has become a very encouraging solution for adaptive control systems. In contrast to traditional control methods, reinforcement learning helps an agent to utilize the interaction with the environment to learn the optimal control policies. This is achieved by the feedback that is received in the form of penalties or rewards, this feedback helps the reinforcement agent to iteratively improve the strategy being used to get the maximum long-term cumulative rewards. The ability of an agent to adapt and learn is what makes reinforcement learning suitable for dynamic and complex systems like HVAC, as there are many factors that dictate the optimal control decisions, including some external factors like, occupancy behaviours, internal heat, and external weather conditions.

1.1 Problem

There are a lot of advantages involved in using reinforcement learning for HVAC control, but with the potential advantages come hindrances to its practical applications. Conventional reinforcement learning algorithms generally use a lot of data and computational resources, this can be very difficult to achieve in real-time situations. HVAC systems have very high dimensional states and action spaces, and this causes complications in the learning process of the model. For this reason, it is necessary to have an efficient algorithm that will perform effectively in real-world situations.

Due to the variability of building environments, there are a few challenges that are experienced in the process of developing scalable and robust reinforcement learning-based HVAC controllers. As mentioned earlier, a few factors like occupancy patterns, architectural design, building size, and insulation properties can come in many variations, and this can make it very difficult for us to generalize reinforcement models across different settings. To encourage

widespread adoption, ensuring that these variations in factors are met by resilience from reinforcement learning controllers and that they can generalize well in the case of varying building conditions and environments.

It is very important to address these above-mentioned issues to reach the maximum potential of reinforcement learning in HVAC systems. The aim of this dissertation is to explore these issues, and this will be done by evaluating and developing a reinforcement learning-based adaptive control model for HVAC systems. There will be a lot of focus on optimizing performance, occupant comfort, and energy efficiency.

1.2 Aims and Objectives

The main aims and objectives of this research are to investigate the circumstances in which reinforcement learning is being applied as related to adaptive control of HVAC systems. This goal is to be achieved by structuring the research around a few specific objectives:

- Developing a Q-learning-based reinforcement learning model.
- Creating a simulated environment.
- Comparison with traditional PID control.
- Evaluating exploration-exploitation strategies
- Proposing enhancements to reinforcement learning algorithms.

1.3 Project Plan

Below is a brief description of the project plan, briefly stating the tasks that would be done in this project.

Developing a Q-learning-based reinforcement learning model: This objective involves implementing a Q-learning algorithm for the maintenance of the temperature settings that is optimal in HVAC systems. This will be done using Jupyter Notebooks and Python to develop the model and the simulation.

Creating a simulated environment: This objective involves the development of a simulated environment to create a room's temperature-like environment, by utilizing factors such as external temperature, insulation, room size, and HVAC power. When there is a need for the evaluation of the performance of the reinforcement learning model, the environment that we have simulated will be the background.

Comparison with traditional PID control: After the reinforcement learning model has been developed and tested, a comparison will be made between the reinforcement learning model and traditional PID control methods. The main bases of evaluation will be the amount of time it takes for response, the amount of energy consumed, and the accuracy of the method.

Evaluating the exploration-exploitation strategies: The exploration-exploitation strategy helps to balance energy efficiency and temperature control. In this research, the effectiveness of these strategies and the influence that these strategies have on the performance of the reinforcement learning model will be evaluated.

Proposing enhancements to reinforcement learning algorithms: This is essential as a very good research work should always leave a gap or possibility for improvements. Therefore, one of the specific objectives of this research is to highlight the advancements that are possible to the reinforcement learning algorithms, to further improve robustness, and to serve as a foundation or reference for future research works in HVAC systems.

It is very important that this research can provide both practical and academic perspectives on energy management and heating, ventilation, and air conditioning controls.

In the practical world, this research will serve as a guide for designing and the application of smart heating, ventilation, and air conditioning control systems. The incorporation of reinforcement learning models will lead to more efficient HVAC systems in homes, which means less energy cost and a more regulated room temperature for houses and the occupants.

Academically, this dissertation will provide valuable contributions to the optimization of reinforcement learning algorithms and their involvement in real-time and real-world situations.

1.4 Dissertation Outline

In this section, each section will be introduced briefly to give a little context into what each section of this dissertation will be about. The aim is to make this dissertation as comprehensive as possible regarding reinforcement learning for adaptive HVAC control. The sections of this dissertation are structured as follows:

- **Introduction** – Chapter 1 describes the overview of the research background, defines the research problem, outlines the main objectives and aims of this research work, expatiates the importance of this project work, and identifies the potential contributions that can be achieved in this research.
- **Literature Review** – Chapter 2 being the literature review chapter is a critical part of the dissertation as it provides a comprehensive general review of research related to this project. This section explains the expectations when it comes to the development of an RL-based HVAC control model, discusses the main concepts and theories that support this research,

and discusses findings, methodologies, and relevance, using qualitative, quantitative, or mixed approaches for a review of the different approach methods used in this research.

- **Methodology** – Chapter 3 in this dissertation will be methodology, this section explains how the RL-based model was created for HVAC control. The way the simulated environment was designed will also be described in this section, and the concept of data generation will also be described.
- **Results and Analysis** – Chapter 4 in this dissertation will be the results and analysis section, this section will dive into the aftermath and outcome derived from the performance of the reinforcement learning model in the environment that was simulated for this experiment. There will be an essential comparison between the reinforcement model and traditional PID control methods. Presents the results of the RL model's performance in the simulated environment, comparing it with traditional PID control methods. Furthermore, an analysis of how effective different exploration-exploitation strategies are will be carried out, highlighting important innuendos.
- **Discussion** – This will be chapter 5 of this dissertation, and the main aim of this part is to discuss the results obtained from this project. The weaknesses and the strengths of the project will be discussed, and the factors responsible for limiting the performance of the reinforcement learning-based control models will also be discussed. In this project, a few challenging obstacles are expected in terms of implementing reinforcement learning models in real-world heating, ventilating, and air conditioning systems, these will also be discussed in this section.
- **Conclusion –** This is the last chapter of this dissertation, and its purpose is to create a summarization of the contributions that this project brings, together with creating a clear

direction for future contributions to research works like this. This is very important because there is a very wide impact of reinforcement learning based HVAC control on the sustainability and the efficiency of energy, and this section of the dissertation helps to draw more attention to this.

2. BACKGROUND

2.1 Literature Review

In this dissertation, there are 2 areas of concentration when it comes to HVAC control methods, the first area of concentration dives into reinforcement learning HVAC control methods, while the second area of concentration dives into traditional HVAC control methods.

2.1.1 Area 1: Reinforcement Learning HVAC Control Methods

This section will discuss the focus of this dissertation, which is reinforcement learning. In layman's language, reinforcement learning is a learning process aimed at obtaining the optimal behaviour in a static or dynamic environment to obtain the maximum reward possible. Reinforcement learning is being used a lot more nowadays as many weaknesses that using PID controllers portray are as many strengths that using reinforcement learning offers. Adaptability is one of the major reasons reinforcement learning is preferred over PID controllers. Reinforcement learning uses feedback from interacting directly with the environment to learn what action to take next to receive the maximum reward while going towards the end goal. In reinforcement learning, the framework can be defined in terms of rewards, actions and states. This is very

effective for interaction between an agent and its environment to achieve a goal despite the presence of uncertainties, external influencing factors and dynamic environments. In [1], reinforcement learning problems are described as closed-loop problems and this is because the action of the learning system affects the subsequent inputs. A very common challenge that is faced when using reinforcement learning methods for is the trade-off between exploitation and exploration. They are two very important concepts when it comes to searching for the final goal, the effective combination of these two approaches provides the most optimal result for a search algorithm. In this project, various exploration strategies will be used to address the exploration-exploitation trade-offs effectively. While it is not always necessary to use value functions for solving reinforcement problems, for this project, value functions will be used for solving this reinforcement learning problem. In this project, the Q-function which is an example of a value function will be used in Q-learning for long-term reward estimations. In terms of comparing traditional PID control with RL-based adaptive control, value functions will be useful in quantifying the advantages of reinforcement learning in terms of response time and energy efficiency, providing a numerical representation of how effective reinforcement learning can be in HVAC system control.

There are various applications of reinforcement learning in today's world, a few of them are for decision-making in uncertain environments, in autonomous robots and as adaptive controllers in industrial processes. In [2], A very impressive application of reinforcement learning was discussed, the program used is called TD-Gammon. As the name suggests it was used for a game of backgammon and surprisingly, the program required very little knowledge of backgammon, yet it learned to play extremely well. It competed with the world's strongest grandmasters. Another impressive application of reinforcement learning according to [2], is in elevator

dispatching. In this application, reinforcement learning was used to decide the order in which elevators should respond to requests especially when there are several requests from different floors. In this project, reinforcement learning is applied in the control of Heating, Ventilation, and Air Conditioning systems in buildings. According to [3], in 2022, 16.4% of global final energy consumption was down to HVAC systems. Most of the energy consumed was from the combustion of fossil fuels which in turn meant more global operational CO_2 emissions of about 14%. With the help of reinforcement learning for HVAC control, more effective use and consumption of energy can be obtained which will reduce the number of emissions of greenhouse gasses contributing to global warming. Before the introduction of reinforcement learning for HVAC system control, there has been the use of PID controllers and despite their advantages in terms of simplicity and cost-effectiveness, they do fall behind when it comes to forward-looking decision-making. For this reason, the introduction of Model Predictive Control was born. The flexibility and the quick response time of Model Predictive Control in handling dynamic situations make it perfect for optimizing HVAC system control, although there is a setback when it comes to learning from previous experiences or consequences. This is where reinforcement learning comes into the picture. Reinforcement learning is based on learning through interaction with the system environment, with this additional characteristic, reinforcement learning can learn how to overcome any challenge that has been faced by previous controllers through adaptation.

In [3], it is evident that even though reinforcement learning is popular for its ability to learn in real-time, without relying on specific pre-defined model parameters, it falls short in this aspect when it comes to HVAC system control. There are different problems like the time it takes for optimizing of the controller when used in real buildings, the discomfort it brings to the occupants

of the buildings, the potential damage that may be caused to the buildings and so on. Therefore, in the case of HVAC system control, the advantages of reinforcement learning over Model Predictive Control are greatly reduced. The advantage of reinforcement learning over MPC is greatly reduced to its ability to make use of experiences from scenarios. When it comes to handling unforeseen occurrences or disturbances like variations in occupancy patterns, sudden weather changes or any other condition that was not encountered during training, there may be a need for fine-tuning, which is very demanding timewise and computationally. To overcome a few of the problems mentioned above, a simulated training environment can be used while considering disturbances and accurately mimicking the target environment. This ensures that the training process is completely risk-free before its application in actual buildings. To ensure that this model is robust enough to handle unforeseen circumstances, exploitation-exploration strategies will be used effectively. A very good way this technique can be implemented is by using dynamic exploration, which means that whenever an unexpected change is encountered, the model will be allowed to explore, and whenever the system is stable, the model will exploit the already-known paths. Improving the models' adaptation and reducing the computational demand necessary for adaptability enables reinforcement learning to outperform the Model Predictive Controller for HVAC system control.

Below is the image of the general concept of a reinforcement learning agent controlling an HVAC system [4].

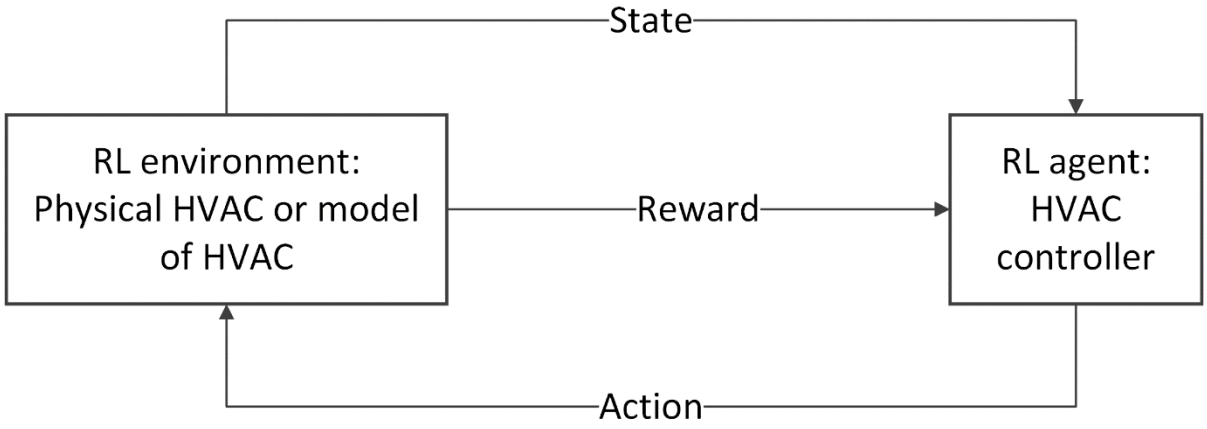

Figure 1: Representation of an RL Agent Controlling an HVAC System

In the above figure, the reinforcement learning agent is the main controller, which controls the system by interacting with the reinforcement learning environment by confirming the state and the reward and following up with an action on the environment. The state and reward information acts as input to the agent, while the action is the output. The state of the environment in the case of this project is temperature, pressure, occupancy data and so on. The serves as another sensor that returns either a positive or negative feedback to indicate the effect that the previous action had on the environment. The agent is then trained to take future actions based on the feedback received from the environment to take actions that are likely to give better rewards in future. This project focuses on using reinforcement learning in a linear environment. Building a reinforcement learning model for a linear environment for HVAC control is a lot different from building a reinforcement learning model for a non-linear environment. For this project, a reinforcement learning model is built for a linear environment for the following reasons: Building a linear model makes it more straightforward to understand, and this can be beneficial if there is a need to explain the model to a diverse audience. Considering the resources available to carry out this project, developing a linear model is faster and requires fewer computational

resources. Building a linear model can act as a benchmark for validating new methodologies [5]. Using a linear model makes it easier to focus on important aspects of this project like exploration-exploitation trade-offs, unlike nonlinear models that have a lot of complexity in their systems dynamics that require lot more attention. The primary objective of this dissertation is to develop and fine tune reinforcement learning algorithms, a linear model makes it much easier to focus on this objective. Linear models are more often used in educational settings for teaching about the concept of and principles of control systems and reinforcement learning; by using a linear model in this dissertation, it creates additional educational resources for future researchers and students.

Linear models are far fewer complex concepts to grasp, as the inputs and outputs are directly proportional, so in the case of an HVAC system, any change in HVAC settings is directly proportional to changes in temperature. A simplified equation to represent a linear model can be seen below.

$$T_{t+1} = a \cdot T_t + b \cdot H_t + c \tag{1}$$

Nonlinear models on the other hand are more complex concepts, the input and output have complex relationships, and a change in input might bring an unexpected or disproportionate change in the output. A few factors like external temperature influence, thermal inertia, and differences in insulation properties can make the relationship between input and output more complex in a nonlinear model. An equation to represent a nonlinear model can be seen below.

$$T_{t+1} = a \cdot T_t + b \cdot H_t^2 + c \cdot \sin(T_t) + d \tag{2}$$

2.1.2 Area 2: Traditional HVAC Control Methods

In this section, a traditional HVAC control method used widely will be discussed; PID (Proportional-Integral-Derivative) controllers were by far the most common control algorithms before the emergence of other control methods involving reinforcement learning. PID controllers are used to regulate temperature, pressure, speed, and flow using a feedback loop to control the performance of the controller. They are indeed the most stable and the most accurate traditional forms of controllers. The three most important parts of a PID controller are the proportional, integral, and derivative control. The purpose of proportional control is to control the overreaction that a system gives to change in error. The purpose of the integral control is to keep the variable that is being controlled at the control point and to prevent an offset [6]. The purpose of the derivative control in a PID controller is to make the system more stable and dampen oscillation. In most cases, a derivative control is ignored, because a PI controller generally does the job [7].

Proportional, Integral, and Derivative controls can be represented mathematically as,

$$Proportional = KPe(t) \tag{3}$$

$$Integral = KI \int_0^t e(t)dt \tag{4}$$

$$Derivative = KD \frac{de(t)}{d(t)} \tag{5}$$

The very first use of PID control for process control can be dated as far back as the early 20th century, since then PID control has evolved and adapted to numerous technological changes. Today, almost all PID controllers are based around microprocessors, for this reason, it has become more and more easier for additional features to be provided. Features like continuous adaptation, automatic tuning, and gain scheduling have all been part of the evolutional features. According to [8], a few out of many applications include; the regulation of variables such as

temperature, flow, and pressure in manufacturing processes, speed and position control in robotic arms and CNC machines, management of current and voltage in inverters and converters, used in automation for production lines to increase productivity, useful in technologies like self-driving cars, autonomous robots for repetitive tasks and UAVs. With the above-mentioned applications of PID control, it is easy to see how crucial it has become in our everyday lives.

In [9], the applications of PID in HVAC systems were discussed. In this article, the author describes a project about improving the energy efficiency of HVAC systems in office buildings by the use of simulation and modelling tools. The HVAC system which is controlled by PID controllers was divided into different control zones. The author convincingly showed how PID controllers were effectively used to regulate electro-valves in the HVAC system of the building, thereby making sure comfort was achieved in terms of temperature control, and efficiency was achieved in terms of energy cost. A main PLC is used to manage all devices while ensuring that there is a net zero balance in terms of energy in the building.

Despite these advantages and the importance of using PID controllers for HVAC control systems, they do have their own limitations, and these limitations are most related to uncertain and dynamic environments. In [10], a few of these limitations were discussed.

Traditional PID controllers find it hard to adapt to an environment that is complex and nonlinear. Traditional PID controllers are built to work well in linear systems and find it hard to adapt when there is nonlinearity in HVAC operations. An HVAC system generally operates in an environment that experiences change easily due to changes in factors like occupancy, and temperature. PIDs work on fixed parameters, and this is why they struggle when there are changing parameters in their new environments. The tuning of the parameters of a PID controller (proportional, integral, and derivative gains) takes a lot of time, and there is no guarantee that

their performance will be satisfactory. In the process of trying to adapt to changes in conditions, it may cause negligible energy use which defeats the aim of energy efficiency. Finally, the unpredictability of disturbances may cause the PID controllers to react slowly, causing a certain degree of inefficiency.

3. MODEL ARCHITECTURE

3.1 Model Description

For this project, reinforcement learning models have been developed for the regulation of and maintenance of a comfortable temperature level, while aiming to optimize energy usage. Different methods have been used to develop these reinforcement learning models. The first method uses Q-learning (with and without randomness), the second method uses double Q-learning, the third method uses deep Q-learning (DQN), and a final comparison will be made using the traditional PID method. This section provides insight into the different algorithms used in these models, as well as their structures and components.

Q-learning:

Environment:

An environment has been created for this project to simulate the temperature of a room equipped with an HVAC system. For this simulation, a few affecting factors were taken into consideration. The state variables for this environment were defined, and factors like HVAC power, room size, room temperature, insulation level and external temperature were used to simulate the ideal HVAC environment. To determine the set of possible actions that can be taken by the HVAC system, an action space was defined which was important for the increment or reduction of the HVAC power level. The action space was a 5-bin state space that allowed for different actions like significant cooling needed (-2), slight cooling (-1), no action (0), light heating (1), and significant heating (+2). In addition to this, the starting temperature of the room (states) was discretized into 10 bins, the temperature values ranged between 15 and 30, and to create a valid

Q-table for the Q-values, a product of actions and space was equal to the table size for storing the Q-values.

$$Q \cdot table\ Size = states(10)\ x\ actions(5) \tag{6}$$

For this model reward functions were defined, and the purpose was to help balance maintaining comfortable temperature levels with energy consumption. For each episode, the agent aims to keep the room temperature as close as possible to the desired room temperature, this process is controlled by the reward function. The mathematical representation of the reward function for this model can be seen below.

$$reward = \begin{cases} 10 & if |T_{room} - 22| < 0.5 \\ 5 & if\ 0.5 \leq |T_{room} - 22| < 1.0 \\ -\min(|T_{room} - 22|, 20) & otherwise \end{cases} \tag{7}$$

The function above ensures that deviations from the desired temperature are penalized while the agent is rewarded for moving closer to the desired temperature.

The Learning Algorithm:

For the learning algorithm, Q-Learning was implemented to train the agent to discover the optimal policy for temperature control in the HVAC system. Important aspects of the algorithm include:

- Exploration strategies: Using the epsilon greedy approach, a balance is struck between exploitation and exploration. The main idea is that most times, a greedy action should be taken, but occasionally a random action should be explored. A random action with probability epsilon (ϵ) should be taken for exploration, and action with the highest Q-value with probability (1-ϵ) should be taken for exploitation. The more the agent learns, the more epsilon decays to reduce exploration.

- Q-function:

$$Q(s,a) \leftarrow Q(s,a) \; + \; \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right] \tag{8}$$

In the above equation, s represents the current state of the environment, a represents the action taken by the agent, α represents the learning rate of the agent, r represents the reward received by the agent because of the action taken, γ is the discount factor, and s' is the next state of the environment.

The Training Process:

Three important aspects of the training process are involved in developing this model. The first is data generation, the second is the training loop, and the third is hyperparameters.

For data generation, interaction is established with the simulated environment, and with a few numerical data like the next state, reward, action, and state, the Q-table is updated.

The training loop is created using 1,000,000 episodes, and in each episode, there are numerous steps, that are executed in a loop until the episode ends or the target temperature is reached. The steps involve initializing the state, choosing an action based on the policy, executing the action observing the reward and the next state, and updating the Q-values by following the Q-learning update rule. These steps are repeated until the training is done or the episode is over.

For the hyperparameters, a learning rate (α) of 0.05 was used, a discount factor (γ) of 0.9 was used, an initial exploration rate (ϵ) of 1.0 was used, an exploration decay rate (ϵ_{decay}) of 0.995 was used, and a minimum exploration rate (ϵ_{min}) of 0.01 was used.

Q-learning (With Randomness):

Environment:

Like the first model, this model has been developed to control the temperature of a room in an HVAC system using Q-learning. This model was tweaked to experiment with the effect of occupancy variability and randomness in the time of the day. For the definition of the state space, 10 discrete temperature bins were created to represent temperatures between 15 degrees and 30 degrees and for discretization, 24 discrete time bins were created to represent 24 hours in a day and 2 occupancy bins were created to represent an occupied and an unoccupied room. An action space was defined, to create 5 discrete actions ranging from heating to cooling. Just like the first model, there are rewards given when the temperature is closer to 22 degrees (for an occupied room) and 18 degrees (for an unoccupied room), and penalties are given when the temperature moves away from the target temperatures.

The Architecture:

For this model, the architecture includes the input layer, Q-table, the hyperparameters, and the learning algorithm.

The input layer is a 3-dimensional state space that is formed as a combination of the temperature bin, time bin and occupancy status.

The Q-table containing the state space dimensions and the action space size is a 3D NumPy array with dimensions "(10, 24, 2, 5)", 10 representing the temperature bin, 24 representing the time bin, 2 representing the occupancy status and 5 representing the action taken in terms of temperature adjustment.

For the hyperparameters, a learning rate (α) of 0.05 was used and the importance of the learning rate is to determine how much new information gained from the learning process replaces old information, the smaller the learning rate, the more conservative the updates. The discount factor (γ) which measures the importance of future rewards was set at 0.9, For the control of the trade-

off between exploration and exploitation, an exploration rate (ε) was set to an initial value of 1.0 to decay to 0.01, the epsilon decay was set at 0.995 and for unoccupied rooms, the decay was faster, which meant a quicker exploration of already learnt strategies, as a way to control the rate at which epsilon decays, the replay memory size was set at 10,000, this was done in an attempt to manage memory size and it indicates that a maximum of 10,000 experiences should be stored, and finally the batch size was set at 64 to indicate that a total of 64 experiences can be selected at random to be used in a training iteration for updating the Q-table. The result of this model was evaluated for unoccupied rooms and occupied rooms separately.

The Training Process:

Using 1,000,000 training episodes again, this model was trained by taking actions chosen randomly using probability epsilon(ε), otherwise, an action with the highest Q-value is selected. The Q-value is updated the same way as the model above, it is updated by the Bellman equation which works by updating Q-values based on the reward received from previous actions, and the maximum expected reward from future actions. As previously explained, experience replay is very essential in developing this learning algorithm, as it is used for storing previous experiences, and creates the opportunity for sampling random batches to be used for updating the Q-table, this can significantly improve the learning stability of the model.

Double Q-learning:

Environment:

This model was used for the regulation of the temperature of a room based on the occupancy status and external temperature of the environment for both during winter and summer. The size of the room, insulation, HVAC power, and external temperature were the same as in the models above. In the action space, the actions that can be taken range from 0 to 4 which represents

different levels of HVAC adjustment and has 5 different bins. The states of the “current temperature”, “time of the day”, and “occupancy status” were discretized into bins, the temperature was divided into 10 bins between 15 and 30 degrees, 2 bins were used to indicate “occupied” and “not occupied” for the “occupancy status”, and 24 bins were created to represent each hour of the day for “time of the day”. Just like in the models above, a high reward was given whenever the deviation of the room temperature from the target was less than 0.5, a medium reward was given when the deviation was between 0.5 and 1, and a negative reward was given for deviations higher than 1. The time of the day constantly increased by an hour, while occupancy status was made to change randomly.

The Architecture:

As the name of this model suggests, there are 2 different learning processes involved in this model, and they are repeated for both the winter and summer seasons. When updating the first Q-table, the actions are selected Using the second Q-table as a reference, likewise, when updating the second Q-table, the actions are selected using the first Q-table as a reference. The temperature bins were divided into 10, with 0 representing the first bin and for values between 15 to 16.5 degrees, until bin 9 which had values between 28.5 to 30 degrees, each bin in between held temperature values with a range of 1.5 degrees.

Just like in the previous model, The input layer is a 3-dimensional state space that is formed as a combination of the temperature bin, time bin and occupancy status.

The Q-table containing the state space dimensions and the action space size is a 3D NumPy array with dimensions “(10, 24, 2, 5)”, 10 representing the temperature bin, 24 representing the time bin, 2 representing the occupancy status and 5 representing the action taken in terms of temperature adjustment.

The learning parameters include a learning rate (α) of 0.05 which was used to determine how much new information gained from the learning process replaces old information, a discount factor (γ) of 0.9, an exploration rate (ε) that was set to an initial value of 1.0 and made to decay to a minimum value of 0.01, and an epsilon decay factor of 0.995.

The Training Process:

This model was trained over 10,000 episodes, and just like in the models above, the agent was made to take randomly chosen actions using probability epsilon (ε). Otherwise, actions with the highest Q-value from the other Q-table were selected. This model has been trained to regulate the temperature of a room both during winter and summer. The aim was to understand how well the double Q-learning model would perform in different weather conditions.

RL vs PID:

Environment:

For the reinforcement learning model and the traditional PID controller comparison to be established, a simulated HVAC system environment was created to model the temperature dynamics of a room. Like previous models, factors like insulation, external temperature, HVAC power, and room size were considered for this simulation. In addition, some actions are used to adjust the internal temperature of the room. The ability of the environment to discretize the state space by binning the time of the day and temperature into bins allows the agent to be able to perform well in a simplified representation of the environment. Just like in other models, there is randomness involved in the occupancy status, and this causes constant changes in the target temperature, it changes between 22 (when occupied) and 18 degrees (when not occupied). The reward function is very similar to the previous models, as it is designed to penalize deviations from the target temperatures and reward closer proximity to the target temperatures.

The Architecture:

The architecture for this comparison was divided into 2, the code had a reinforcement learning part, and the second part was for the traditional PID controller. In the reinforcement learning model (Q-learning), the agent does a good job balancing exploration and exploitation using an epsilon-greedy strategy, and just like previous models, either a random action (Exploration) is selected or an action with the highest estimated Q-value (Exploitation) is selected. The experience of the agent is updated using the values stored in the Q-table. The Q-table is updated with the use of experience replays which are used to store recent experiences in memory. Another exploration-exploitation strategy used for this comparison is the SoftMax approach as it allows for a smoother transition into exploitation as the Q-values for the best actions increase. A traditional PID controller in this code is used as the benchmark, the error between the current temperature and the desired temperature is used to compute the HVAC action, and the `proportional, integral and derivative gains were tuned using the Ziegler-Nichols tuning method.

The Training Process:

The training process for this comparison also involves 2 parts, the reinforcement learning model, and the traditional PID controller. For the RL model, the training process was not so much different from the previous models, as it involved running episodes with the agent interacting with the environment. Each episode starts with predefined or random initial temperatures, and the job of the agent is to keep the temperature as close as possible to the target temperatures, which were either 22 or 18 degrees according to the occupancy status of the room. The Q-learning update rule is then used to update the Q-table. The learning process starts initially with more exploration and gradually exploits more as the training progresses, this is done over 1,000,000 episodes, and the performance of this model is evaluated using energy consumption,

accuracy, and response time. As stated above, the Ziegler-Nichols tuning is used to tune the PID controller, this is very important before training, and it is done through the estimation of the ultimate gain and the oscillation period. After the tuning, the PID controller maintains the temperature of the environment by using its continuous control method. Just like in the RL training process, the performance is measured.

Deep Q-Network:

Environment:

The environment of the deep Q-Network model is like that of the previous models, in a similar fashion, an environment with core parameters like room size, HVAC power, insulation, and external temperature was simulated, and the temperature of the room was made equal to the external temperature, and the agent took actions to increase or decrease the temperature of the room. For this model, the negative reward was capped at -50, to prevent extreme values for penalization. All other environmental characteristics were the same as in previous models.

The Architecture:

The architecture of this model is a bit more complex than the previous models, as this model combines Q-learning with neural networks. Neural networks are known for their multiple-layered approach to solving a problem. Therefore, the architecture of this model consists of the input layer, two hidden layers, and the output layer. The input is the state of the environment, which is defined as the current temperature of the room, and it is a 1-dimensional value (temperature bin). The two hidden layers serve the purpose of helping the network learn complex relationships between the optimal action and the temperature, it consists of 32 neurons and a ReLU activation function. The output layer consists of 5 neurons which represent the action space size, as they are the possible actions the agent can take, the output of this model is Q-

values for each action, and the agent uses these Q-values to decide what action will yield the highest possible reward.

The Training Process:

The training process of this model makes it so reliable, as the network is trained using the Mean Squared Error loss function, as seen in equation (10), the training is done by minimizing the difference between the target Q-values and the predicted Q-values, in this case, the target Q-values are computed using bellman equation like in previous models, and the predicted Q-values are from the network. The optimizer used for this training is called Adam, and it is very popular when it comes to training neural networks.

The training is done in loops through episodes, and this model was trained in over 10,000 episodes, and this took an approximate of 6 days to finally be completed. Each episode took about an hour each, or in some cases earlier and the cycle was repeated 10,000 times.

The training loop is explained below:

1. The environment is reset at the beginning of every episode
2. An action is selected using the epsilon greedy policy as explained in previous models
3. According to the selected action, the new temperature is calculated, and a reward is returned.
4. The experience is stored in the agent's memory
5. Once the agent has gathered enough memory, the agent begins to train using the experience replay that has been defined in the code. The experiences are selected randomly, and if the goal is reached the target is going to be the reward received, if not, then the target is going to be the reward and the discounted maximum future reward.

$$y = r + \gamma \cdot \max_{a'} Q(s', a') \tag{9}$$

$$Loss = \frac{1}{N}\sum_{i=1}^{N}(y_i - Q(s_i, a_i))^2 \quad (10)$$

6. Just like in previous models, there is an epsilon decay after every episode to reduce the rate at which the agent explores and allow to agent to exploit more using the highest possible Q-values gained from previous experience.

Due to the long time required to train this model, checkpoints have been defined to ensure that the progress of this training is being saved in case of unexpected errors or system crashes. Finally, stopping criteria have also been defined to prevent overtraining by ensuring that the training stops when the average rewards in recent episodes exceed a certain threshold.

3.2 Model Analysis

This section gives a deep insight into the different models' performances, using theoretical and empirical methods. In this section, the steps taken to obtain the most accurate models possible, subject to the available resources are fully explained. In each episode, random initial room temperatures are initialized, and using the epsilon greedy policy, various actions are taken. As epsilon decays, exploration is reduced over time, and the Q-table is updated. Four different approaches have been taken in this project, using 3 different control techniques. 2 different models under different conditions have been trained using Q-learning, and another model has been trained using DQN (Deep Q-learning) and traditional PID methods for temperature control in an HVAC system have been experimented.

Q-learning:

The first reinforcement learning model that was trained learnt using Q-learning without the dynamic occupancy factor. As stated in previous sections, the model trained over 1,000,000

episodes and the performance was recorded in terms of time taken to complete, average rewards, and episode lengths.

The room environment used in the process of training this model has the following dynamics:

Room size: 50 square meters

Insulation: 0.1

External Temperature: 20°C

HVAC Power: 5

A room size of 50 square meters was chosen to provide the perfect balance between a large room and a small room, not too large to keep the simulation computationally manageable, and not too small for the observation of significant temperature dynamics.

An insulation of 0.1 was chosen to represent a moderately insulated room so that HVAC actions can cause notable temperature fluctuations. The aim is to create a challenging environment for the algorithm to learn effective control strategies.

An external temperature of 20°C was chosen because it's not extremely hot, nor is it extremely cold.

An HVAC power of 5 was chosen to make sure that the HVAC system can effectively alter the room temperature, which is the case in almost all buildings.

The action and dynamics of the environment determine what the current temperature will be. The agent will be rewarded for keeping the room temperature close to the target temperature and penalized for moving further from the target temperature.

For the evaluation of the performance of the reinforcement learning model using Q-learning, a few pieces of information were obtained from the training model, information like a maximum

reward, minimum reward, average episode length, overall average reward, maximum episode length and minimum episode length was used for evaluating the performance of the model.

Q-learning (With Randomness):

The second reinforcement learning model was also trained using Q-learning with the introduction of a dynamic property into the environment. The dynamic property that was introduced was the occupancy factor. Just 2 bins were created to represent “occupied” and “not occupied”. In the results section, the effect that this additional factor had on training the reinforcement learning model will be examined. This model was also trained for 1,000,000 episodes and just like the previous model, the performance was recorded in terms of time taken to complete, average rewards, and episode lengths.

The room environment used in the training process of this reinforcement learning model is identical to the environment that was used for the previous model, and it has the following dynamics:

Room size: 50 square meters

Insulation: 0.1

External Temperature: 20°C

HVAC Power: 5

Time of Day: Any random time in 24 hours

Occupancy: Occupied or unoccupied

Regarding the time of the day factor, 24 bins were created to create space for different hours of the day. This steady increment of the hours of the day affects the external temperature and the

occupancy, as temperatures tend to drop later, and a room will be either occupied or non-occupied depending on the time of the day. Additionally, the target temperature is changed to 18°C when it is not occupied and to 22°C when it is occupied.

Occupancy status is the main effecting variable in this environment and the training of this model, this is because there is a huge shift in target temperature in the environment from 22°C to 18°C as soon as there are no occupants in the room and vice-versa as soon as an occupant steps into the room. This helps to create a more complex environment, which can be more advantageous in more complex non-simulated environments. In this environment, occupied is represented as (1) and unoccupied is represented as (0). The main idea behind changing the target temperature to 18°C when there are no occupants in the room is to save energy. It is no news that less energy is used when the temperature is set lower, so with this technique, it is possible to further maximize energy saving by not always trying to reach the higher temperature of 22°C but also targeting lesser temperatures like 18°C when the room is empty, while at the same time accomplishing another aim which is occupant comfortability. At the end of the training, information gathered from the outcome has been used to evaluate the performance of the model. Vital information like maximum reward, minimum reward, average episode length, overall average reward, maximum episode length and minimum episode length were used for evaluating the performance of the learning model.

Double Q-learning:

As stated in the previous section, this model was trained using double Q-learning with the use of a dynamic property called “occupancy” and was trained for both winter and summer seasons to measure how well double Q-learning performs when there are extreme temperatures in unknown environments. Just 2 bins were created to represent “occupied” and “not occupied” in the case of

the occupancy varying factor. In the results section, the effect that this additional factor had on training the reinforcement learning model will be examined. This model was also trained for 10,000 episodes, unlike the Q-learning models that needed to be trained for 1,000,000 episodes before a positive value for average reward was attained. An indication that a model has learnt well is a reward value of 0 and above. Just like the previous models, the performance of these models was recorded in terms of time taken to complete, average rewards, and episode lengths.

The room environment used in the training process of this reinforcement learning model is identical to the environment that was used for the previous model, and it has the following dynamics:

Room size: 50 square meters

Insulation: 0.1

External Temperature: This value was not random, but it varied a lot to make it possible for efficient training of the model in different temperature conditions and seasons.

HVAC Power: 5

Time of Day: The time of the day increased during the training, from 0 to 23

Occupancy: Occupied or unoccupied

Just like the models above, 24 bins were created to create space for all the hours of the day. These steady increments affect the external temperature and the occupancy, as temperatures tend to drop later, and a room will be either occupied or non-occupied depending on the time of the day. Additionally, the target temperature is changed to 18°C when it is not occupied and to 22°C when it is occupied.

RL vs PID:

The reinforcement learning model of this model comparison was trained using Q-learning, with the use of different newly introduced factors for a bit of dynamism and complexity in the environment. Some of the added factors include randomly changing temperatures, random occupancy status, and random memory sampling of experience, this model was also made to train over 1,000,000 episodes, and the performance was evaluated in terms of accuracy, response time, and energy consumption. There are also graphical visualizations to clearly show the performance of the reinforcement learning model and the PID Controller.

The room environment used in the process of training this model has the following dynamics:

Room size: 50 square meters

Insulation: 0.1

External Temperature: 20°C

HVAC Power: 5

In this environment, random initial temperature was used to see how the RL model and the PID controller handles situations that are close to a realistic HVAC system. Just like the other models, the agent is rewarded for keeping the room temperature close to the target temperature and penalized for moving further from the target temperature.

Deep Q-Network:

This is the last reinforcement learning model that was used to train an agent to control the temperature in an HVAC system. There was no randomness in this environment as the initial temperature was fixed, and the occupancy status was fixed. The model trained over 10,000 episodes and it took a total of 6 days to be completed successfully. The performance of the model was recorded in terms of time taken to complete, average rewards, and episode lengths.

The room environment used in the process of training this model was the same as the initial Q-learning model without randomness:

Room size: 50 square meters

Insulation: 0.1

External Temperature: 20°C

HVAC Power: 5

This model was computationally demanding, as it is trained using neural networks where there are different layers involved including hidden layers that allow the model to learn complex patterns and relationships between the optimal actions and the state of the environment. Just like in other models, a significant reward was given when the room temperature came close to the target temperature, and in other instances where the deviation was more than 0.5, penalties were given. This agent learns from experience and updates its Q-values based on the previous actions. Due to the complexity and the time taken to successfully complete the training of this model, the performance was monitored by periodically evaluating the mean reward of every 100 episodes. If the performance of the agent exceeds the set threshold, then the training process is stopped early. Epsilon decay as mentioned earlier is used to balance exploration and exploitation, the agent is initially allowed to explore, to gain diverse experiences about the environment.

The major performance metrics are episode reward and episode length. Episode reward is used to measure how well the agent has kept the temperature close to the target temperature, and the episode length is used to measure the efficiency of the model by tracking how many steps it took to reach the target temperature. Model checkpoints were introduced after the model crashed unexpectedly and the training had to be started all over, the training was repeated but this time

with model checkpoints to allow for recovery in the case of any unexpected interruption, this helps the training to resume without losing progress.

A comparison of the rewards and episode lengths over time is evaluated to evaluate the performance of this reinforcement learning model using DQN. The success of this model is determined by the consistency the agent sticks to a policy that consistently keeps the temperature close to 22 degrees. In the results section, there are visual representations of rewards and episode lengths that provide insights into how well the model is performing in the environment.

3.3 Exploration-Exploitation Strategies

Different exploration-exploitation strategies' effects on maintaining optimal temperature and energy efficiency were evaluated. The reinforcement learning models using Q-learning, Q-learning with occupancy randomness, and Double Q-learning were created using similar environmental conditions and parameters as in the training models. Below is a description of the models that were used to evaluate these strategies:

In each of the models, three strategies were integrated to evaluate how well the balance of exploration and exploitation can be achieved in the reinforcement learning models. The first strategy that was integrated was Epsilon-Greedy, which is focused on taking random actions based on a certain probability (epsilon) to explore, as exploring is the priority in other to discover some paths to the goal that may not be easy to find by just exploitation. Exploitation is gradually introduced to take advantage of the already known paths to yield positive rewards from exploration.

The next strategy that was used is SoftMax, in SoftMax, the Q-values are turned into probabilities for the next action, and these Q-values are then probabilistically used to choose

actions. After enough experience has been gathered, the Q-value gives a good image of how effective certain actions are. The Q-values of a particular state are passed through the exponential function below to scale them, and to make sure that the actions with higher Q-values have higher probabilities, and at the same time make sure that there are no lower-valued actions with zero probability to allow exploration, this is an aspect of exponential scaling.

$$\exp\_q_i = e^{Q_i - \max(Q)} \tag{11}$$

and the probability of selecting action a in state s is given by:

$$\mathrm{P(a|s)} = \frac{e^{\frac{Q(s,a)}{T}}}{\sum_b e^{\frac{Q(s,b)}{T}}} \tag{12}$$

Another important aspect of exponential scaling is normalization, this is very important to ensure that large values are handled effectively. Unlike in the case of Epsilon-Greedy where the action with the highest Q-value is chosen, in SoftMax, Q-values are chosen according to the probabilities assigned to them, which means higher Q-values have higher probabilities, but there is the chance for other actions to also be selected. Another exploration strategy used in this model is Upper Confidence Bound (UCB) for balancing exploration and exploitation, UCB ensures that actions that are not chosen as often as others but have the potential to be optimal are more often chosen to be explored. UCB works by considering both the uncertainty of the agent about an action and the Q-value of the action, in this model, the number of times an action has been chosen in a particular state is tracked and stored, and actions that have been taken more have higher visit counts, while actions that have been explored less have lesser visit counts. For every action that is chosen, the agent computes a UCB (Upper Confidence Bound) which combines two factors, the exploration term and the exploitation term. The exploration term is

what encourages the agent to explore actions that have not been explored so much, while the exploitation term is simply the Q-value for a given action. The UCB equation for each action is:

$$UCB_i = Q_i + \sqrt{\frac{2\ln(N+1)}{n_i+1}} \quad (13)$$

Where:

ln is the natural logarithm

Q_i is the estimated Q value or reward for the action

n_i is the number of visits to the action

N is the total number of visits

UCB_i is the UCB value for the action

1 is added to both N and n_i to avoid any division by zero.

4. EXPERIMENTAL RESULTS

This project investigated different reinforcement learning models for room temperature control on an HVAC system, for this experiment, a few other models were tested, different exploration-exploitation strategies were also evaluated, and there was a comparison between a reinforcement learning and a traditional PID controller. Finally, a deep Q-network reinforcement learning model was implemented to evaluate the effectiveness of neural networks in adaptive control systems.

In this section, the results for the following models will be examined with relevant images to give a clearer perspective on the performances of these models.

4.1 Methodology

Q-Learning:

This model is a Q-Learning agent created to learn the optimal policy to control the temperature of the environment described in section 3.

In this model, the exploration strategy that was used was the epsilon-greedy strategy, and the exploration was set to start at 1.0 and the exploration rate decayed at 0.995 per episode, and was not allowed to go below 0.01, this enabled the agent to gradually shift towards exploiting the action with the highest Q-value as the training progressed.

The environment simulation was done over 1,000,000 episodes, and at the beginning of each episode, the external temperature was set at 20 degrees and the indoor temperature was set equal to the external temperature. Each episode was terminated as soon as the room temperature reached within 0.5 degrees of the target temperature which was 22 degrees after an iterative process of selecting actions, receiving rewards and updating the Q-table.

The Evaluation Criteria:

Q-table analysis was done at the end of the training to visualize how the agent distributed values to different state-action pairs. The heatmap that was produced because of this training can be seen in the results section. Episode rewards and episode lengths are important metrics that were used for the model evaluation, with shorter episode lengths meaning that the agent learnt to regulate temperature more efficiently.

Q-Learning (Exploration-Exploitation Strategies):

The main purpose of this experiment was to investigate the performance of three different strategies of exploration-exploitation strategies in Q-learning. Just like in the previous model, the simulation was done over 1,000,000 episodes and each episode started with an initial room temperature of 20 degrees which is the same as the external temperature. Action is selected based on its exploration-exploitation strategy, as the three strategies implemented in this model are epsilon-greedy, SoftMax, and UCB, the environment responds with a temperature change which is controlled by factors like external temperatures, insulation and HVAC power level. The reward is calculated the same way as the model above.

The Evaluation Criteria:

Just like in the model above, a Q-table was produced to visualize the learned value of taking certain actions in each state. The final Q-table was produced in the form of a heatmap for better visualization. Episode length and episode reward values were also generated to evaluate the performance of the different exploration-exploitation strategies.

Q-Learning with Occupancy Dynamism:

In this model, the goal was to create a more dynamic environment and more realistic environment conditions, occupancy fluctuation was introduced into this model and the target temperature was 22 degrees when the room was occupied and 18 degrees when the room was empty, this is to reduce the load on the HVAC system when the room was empty, as maintaining a lower temperature level requires lesser energy that maintaining a higher temperature level. For this model, the penalty was capped at -50 to prevent extreme values of negative rewards. This model also involved running 1,000,000 episodes, with each episode running for a simulated full day, to make this possible, a time bin was created to create a space for 24 values ranging from 0-23 as a representation of each hour of the day. Each episode of this model began with a random time of the day and the time of the day determined the initial starting temperature of the room. Finally, just like the previous models, actions are selected, the next state is observed, rewards are received, and the Q-table is updated accordingly. For this model, only the epsilon-greedy exploration strategy was used, in unoccupied rooms, and the decay rate was more aggressive to improve energy efficiency in unoccupied rooms.

The Evaluation Criteria:

The performance of the agent in an occupied vs unoccupied room was evaluated separately in terms of the rewards and episode lengths. The reward was also used for evaluating the performance of the model, and a visual representation of how this model performed over time can be seen in the results section below.

Q-Learning with Occupancy Dynamism (Exploration-Exploitation Strategies):

The main purpose of this experiment was to investigate the performance of three different strategies of exploration-exploitation strategies in the Q-learning with occupancy dynamism model. This model was trained over 10,000 episodes, and for each episode, an interaction was

established with the environment by selecting actions according to the exploration-exploitation strategy, and just like other models, the Q-table is updated according to the rewards received. The state of this model is represented by a tuple of (occupancy status, time bin and temperature bin), the occupancy status has two values (occupied and unoccupied), the time bin has 24 values (0-23), and the temperature bin has 10 values between 15 and 30 degrees. Epsilon-greedy, SoftMax and UCB were used to evaluate and compare the effect of exploration-exploitation strategies in this reinforcement learning model.

The Evaluation Criteria:

The performance of each model was evaluated using metrics like average reward, maximum and minimum rewards, total reward per episode, and average episode length. The visual representation of these results can be seen in the results section.

Double Q-Learning:

The model was trained using double Q-learning and separately to investigate the performance of the model in a simulated summer and winter environment, there were 10,000 episodes for this training, with each episode representing a day of 24 hours. The model is constructed, to begin with a random initial temperature and time of the day for each episode, an interaction is then formed with the environment by performing actions like either heating or cooling depending on the state of the environment, with factors like time of the day, occupancy level, and temperature defining the state of the environment at a particular instance. The Q-table is updated using the double Q-learning update rules and experience replay mechanisms.

The Evaluation Criteria:

For the evaluation of the performance of this model, the episode length, episode rewards, and Q-table values were well returned for analysis and comparison for both the summer and winter

seasons. The comparative analysis for the summer and winter training sessions makes it possible to understand how different seasons affected the agent's performance.

Double Q-Learning (Exploration-Exploitation Strategies):

Just like in previous models, this double Q-learning model was evaluated using different exploration-exploitation strategies like SoftMax, epsilon greedy, and UCB. Like other models, there is an initialization stage for each episode, constant interactions with the environment to update the Q-table, and an early termination when the target is successfully maintained at 0.5 degrees Celsius.

The Evaluation Criteria:

During the training, data like episode rewards, episode lengths and Q-values were recorded.

RL vs PID (Epsilon-Greedy):

This model was used to compare the performance of a reinforcement learning model with a traditional PID controller, the exploration-exploitation strategy that was used for this comparison is the epsilon-greedy strategy. For the Q-learning experiment, there is a constant interaction with the HVAC environment over 1,000,000 episodes, at the start of each episode, a new initial state based on the occupancy status, current temperature and the time of the day is initialized, and based on the feedback from the environment, the Q-table is updated. For the PID controllers, the environmental conditions are similar the required action is computed based on the current temperature, and the process is repeated until the target temperature is reached or the maximum episode length is reached.

The Evaluation Criteria:

For the evaluation of this mode, some data like accuracy, energy consumption, and response time are recorded.

RL vs PID (SoftMax):

This model is like the previous model but focuses on SoftMax for Exploration-Exploitation balancing instead of the Epsilon greedy approach, the episode was terminated whenever the model reached within the range of the target temperature, or after a maximum of 100 steps.

The Evaluation Criteria:

Like the model above, the metrics for evaluation are energy consumption, accuracy, and response time. The results are visualized to provide a clear comparison of the performance of the reinforcement learning model with the PID controller.

DQN Agent:

This is a Deep Q-Network model used to control the temperature of a room in an HVAC system, this model was very complex and posed a few problems in the first few trials. The system used for this training was an Asus Vivobook with processor AMD Ryzen 5 with Radeon Graphics and 8GB RAM, the limited GPU memory in this system caused the training to stop on 3 different occasions abruptly. Finally, TensorFlow was configured to manage GPU memory growth dynamically, this allows the program to allocate some memory as needed. The model was designed to operate in a state space size of 1, representing the temperature bin, and an action space of 5 representing the different actions taken on the HVAC power level; the learning rate was set at 0.001 a decay rate of 0.995 starting at an exploration rate of 1.0, a batch size of 128 and a discount factor of 0.95.

Due to the efficient nature of the Deep Q Network agent, and the time taken to complete a training session, the training was done over 10,000 episodes, and a maximum of 200 steps per

episode was defined. Due to the tendency of this model to crash mid-way, checkpoints were saved to allow for easy continuation of the training in the case of any abrupt stop.

The Evaluation Criteria:

The performance of this model was measured by tracking the rewards over time, the memory usage was also tracked over time. There was a major difference between the first few thousand episodes and the last few thousand episodes. These results will be shown in the result section.

4.2 Results

The results of the models explained above can be seen below.

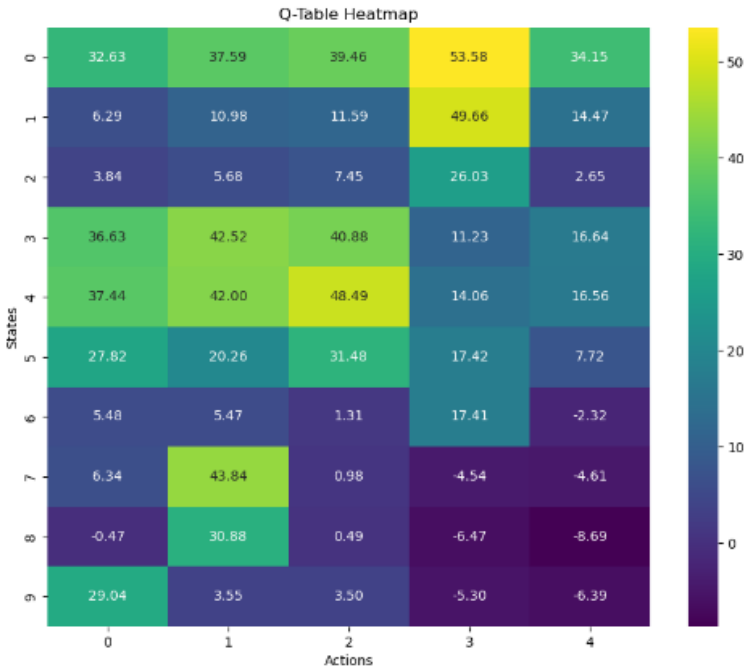

Figure 2: Q-learning model Q-table using epsilon greedy

Figure 3: Q-learning model Q-table using SoftMax

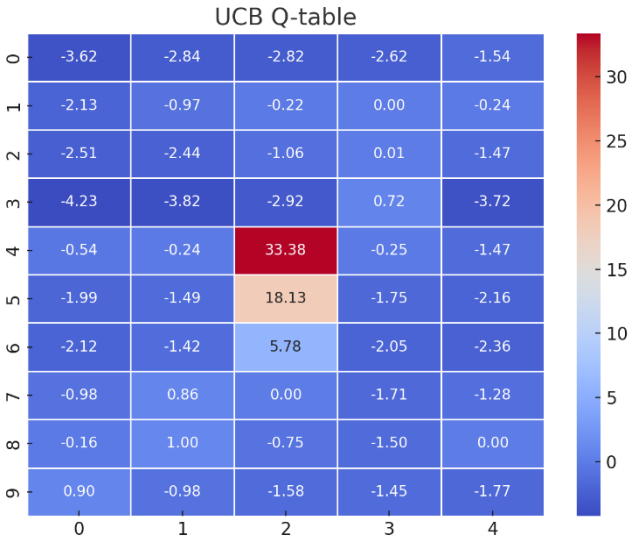

Figure 4: Q-learning model Q-table using UCB

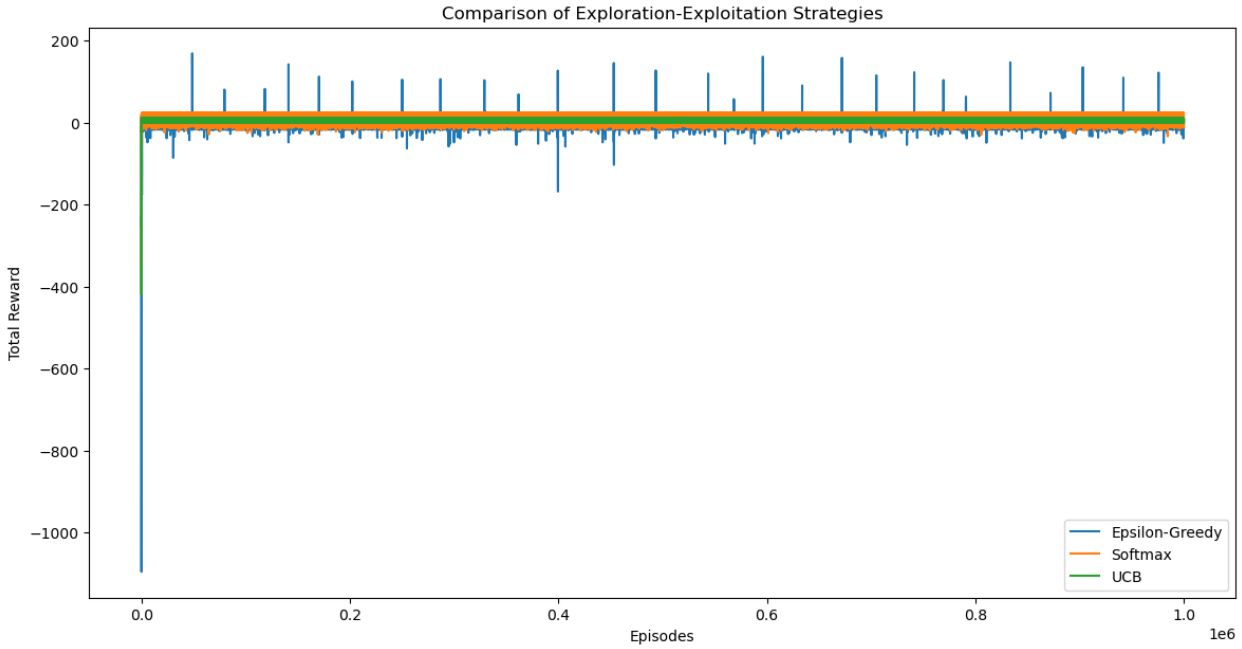

Figure 5: Comparison of Exploration Strategies for Q-learning

	Average Reward	Maximum Reward	Minimum Reward	Average Length	Maximum Episode Length	Minimum Episode Length
Epsilon-Greedy	**7.15**	**168.58**	**-1096.04**	**1.56**	**187**	**1**
SoftMax	**9.86**	**23.22**	**-347.88**	**4.13**	**56**	**1**
UCB	**7.27**	**10**	**-419.43**	**3.5**	**71**	**1**

Table 1: Exploration-Exploitation Strategies Performance For Q-learning

The Q-tables in Fig. 2, 3, and 4, are derived from different exploration strategies in reinforcement learning, it can be seen in Fig. 2 that the values show a broader range, with large

positive values, which shows that the agent did a lot of exploitation of actions with high Q-values but mixed it up with exploration to be able to create a balance between exploration and exploitation, but the model failed to explore all possibilities in certain regions. Fig. 2 is an indication that SoftMax works by placing probabilities on each action based on the Q-values, and the actions with higher Q-values are chosen more frequently, while lower values still have a chance. The Q-values are more centralized, and the highest values are in the middle of the Q-table, with values like 42.67, and 24.95. This suggests that the agent explored more artificially, but the actions were still controlled by Q-values. The UCB Q-table has a much more concentrated pattern, a value of 33.38 at the middle of the Q-table and the presence of many negative values around indicates that the agent found the region with the best possible action and has explored other areas with lower rewards.

In conclusion, epsilon-greedy is a strategy that does a lot of exploitation and can frequently miss out on other actions that need to be explored, SoftMax finds the perfect balance between exploration and exploitation by giving all the values a probability but giving more importance to higher valued actions. UCB has a more aggressive approach and explores a lot more and gives a sense of reliability at the expense of more exploration.

The result of these exploration strategies when a bit of randomness was added to the model can be seen below.

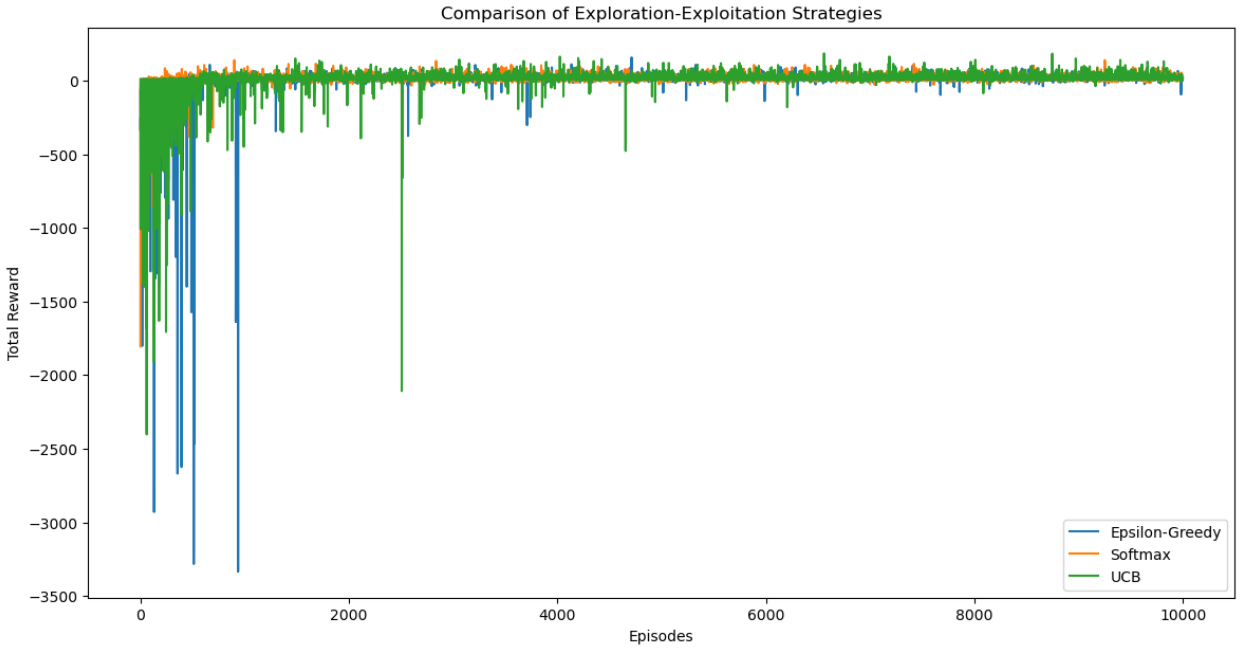

Figure 6: Comparison of Exploration Strategies for Q-Learning with Occupancy Randomness

	Average Reward	**Maximum Reward**	**Minimum Reward**	**Average Length**	**Maximum Episode Length**	**Minimum Episode Length**
Epsilon-Greedy	**5.50**	**157.57**	**-3335.45**	**5.63**	**231**	**1**
SoftMax	**12.16**	**139.31**	**-1805.33**	**5.1344**	**147**	**1**
UCB	**8**	**185.16**	**-2402.82**	**5.2542**	**196**	**1**

Table 2: Exploration-Exploitation Strategies Performance for Q-learning with Occupancy Randomness

Table 2 and Fig. 6 show that the epsilon-greedy strategy has a lot of variety when it comes to rewards and episode lengths, and this is due to how random the exploration approach is.

Sometimes, it can find very good rewards, and some other occasions very poor outcomes. In the case of SoftMax, again it is the best-performing strategy as it does a very good job of balancing exploration and exploitation, and that's why the minimum reward is best, the average and maximum length is the shortest, and the average reward is the highest. UCB once again turned out to be a balance of epsilon-greedy and SoftMax, it performed worse than SoftMax but better than epsilon-greedy. It obtained the highest maximum reward, meaning it has the potential to identify the most optimal action.

Another experiment was carried out on the exploration strategy for the double Q-learning model, and the results can be seen below.

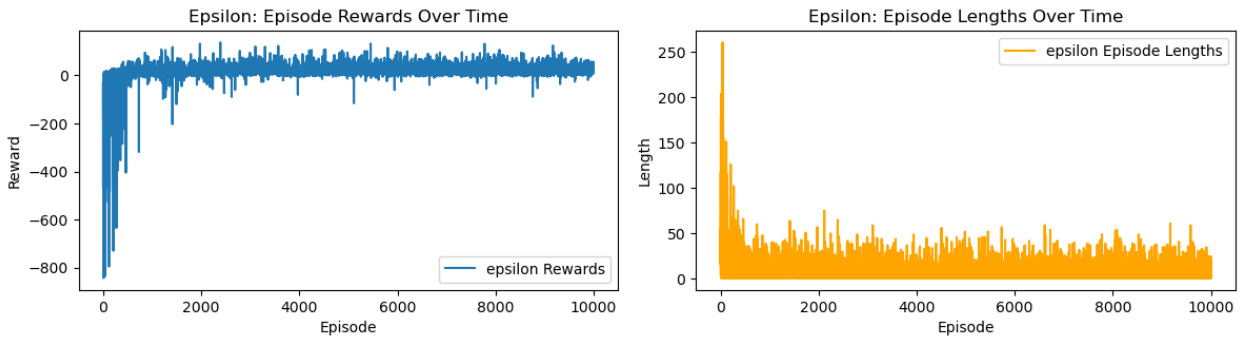

Figure 7: Reward & Length for Epsilon-Greedy

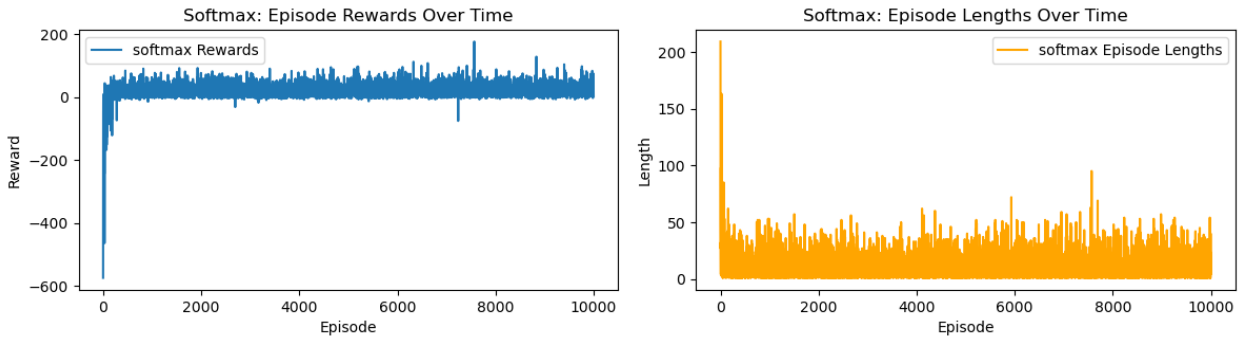

Figure 8: Reward & Length for SoftMax

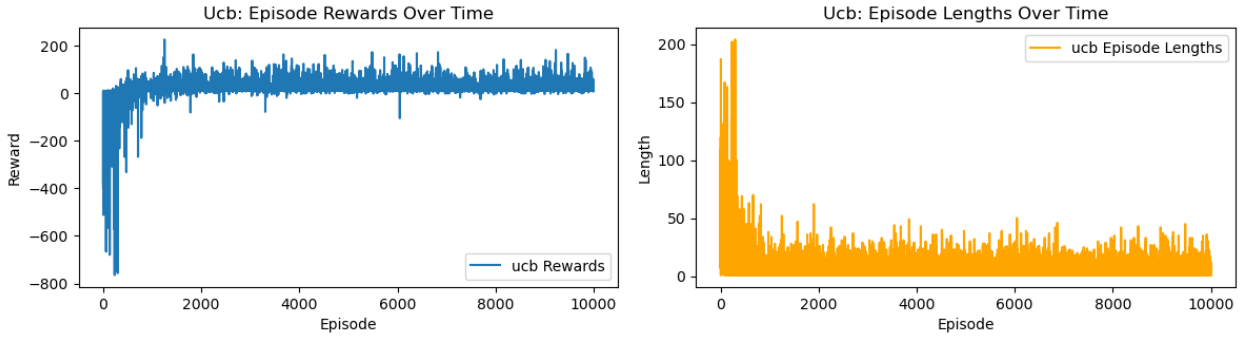

Figure 9: Reward & Length for UCB

	Average Reward	Maximum Reward	Minimum Reward	Average Length	Maximum Episode Length	Minimum Episode Length
Epsilon-Greedy	**16.37**	**137.39**	**-843.85**	**8.73**	**260.00**	**1**
SoftMax	**17.45**	**175.77**	**-574.43**	**10.35**	**209**	**1**
UCB	**18.85**	**225.21**	**-764.23**	**7.18**	**204**	**1**

Table 3: Exploration-Exploitation Strategies Performance for Double Q-Learning

The table above shows that UCB has the best performance in terms of rewards, as UCB has the highest average reward, the most stable strategy seems to be SoftMax as it has the lowest minimum reward out of the three strategies. Finally, in terms of efficiency, UCB comes out on top, with an average episode length of 7.18 and a maximum episode length of 204 which is the least of the three strategies.

In general, UCB seems to perform the best in this Double Q-learning reinforcement learning model, and once again epsilon-greedy performs the worst generally.

Below are the results obtained from the comparison of RL and PID using both epsilon greedy and SoftMax.

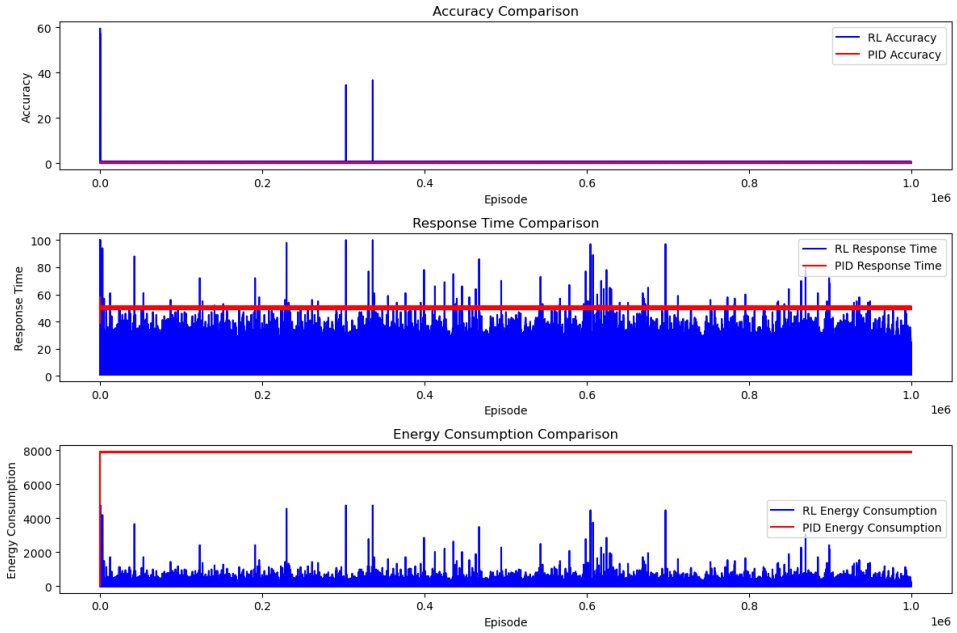

Figure 10: RL vs PID Using Epsilon-Greedy

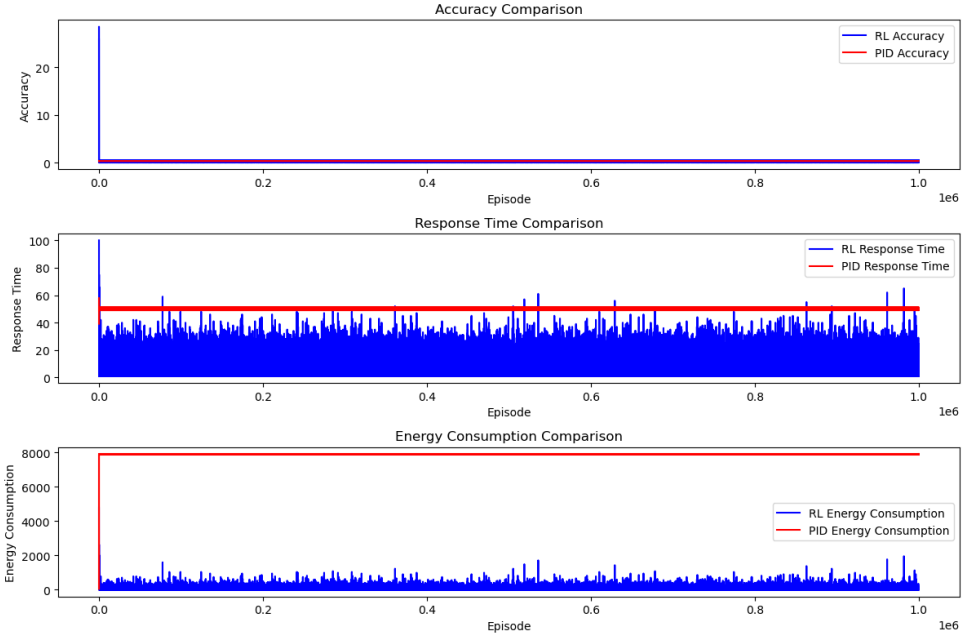

Figure 11: RL vs PID Using SoftMax

	Accuracy	Response Time	Energy Consumption
RL	0.25	4.32	15.42
PID	0.32	50.00	7889.75

Table 4: RL vs PID Using Epsilon-Greedy

	Accuracy	Response Time	Energy Consumption
RL	0.24	4.13	12.75
PID	0.32	50.00	7889.75

Table 5: RL vs PID Using SoftMax

The results from the reinforcement learning model and the PID controller show how badly the PID performs when it comes to energy efficiency. An energy consumption value of 7889.75 is extremely high, indicating that in a very unstable environment like the one that was simulated for this experiment where the initial temperature was rapidly changing, and occupancy status kept changing, the traditional PID approach may not be suitable, the extremely random nature of this environment causes the controller to consume a lot of energy in the process of achieving stability.

Overall, the Reinforcement learning performs between using both exploration strategies, with SoftMax coming out on top once again.

The result to be shown below is the result obtained from the Deep Q-Network model, to show how well this model learned, the first two thousand rewards will be compared with the last 2 thousand rewards.

```
Episode: 0, Recent Mean Reward: -330.75, Memory Usage: 95.0%
Episode: 100, Recent Mean Reward: -595.37, Memory Usage: 84.6%
Episode: 200, Recent Mean Reward: -405.67, Memory Usage: 86.5%
Episode: 300, Recent Mean Reward: -115.09, Memory Usage: 86.6%
Episode: 400, Recent Mean Reward: -58.41, Memory Usage: 83.5%
Episode: 500, Recent Mean Reward: -38.79, Memory Usage: 86.3%
Episode: 600, Recent Mean Reward: -25.53, Memory Usage: 86.9%
Episode: 700, Recent Mean Reward: -8.83, Memory Usage: 86.1%
Episode: 800, Recent Mean Reward: -22.96, Memory Usage: 87.1%
Episode: 900, Recent Mean Reward: -30.67, Memory Usage: 87.0%
Episode: 1000, Recent Mean Reward: -60.63, Memory Usage: 87.5%
Episode: 1100, Recent Mean Reward: -27.06, Memory Usage: 87.5%
Episode: 1200, Recent Mean Reward: 4.68, Memory Usage: 88.5%
Episode: 1300, Recent Mean Reward: 6.99, Memory Usage: 88.8%
Episode: 1400, Recent Mean Reward: 7.04, Memory Usage: 87.6%
Episode: 1500, Recent Mean Reward: 7.00, Memory Usage: 89.3%
Episode: 1600, Recent Mean Reward: 7.27, Memory Usage: 90.6%
Episode: 1700, Recent Mean Reward: 7.21, Memory Usage: 91.2%
Episode: 1800, Recent Mean Reward: 6.69, Memory Usage: 92.2%
Episode: 1900, Recent Mean Reward: 6.93, Memory Usage: 90.2%
Episode: 2000, Recent Mean Reward: 6.38, Memory Usage: 90.8%
```

Figure 12: Performance of the Model for the First 20 Episodes

```
Episode: 7800, Recent Mean Reward: 7.31, Memory Usage: 93.8%
Episode: 7900, Recent Mean Reward: 7.08, Memory Usage: 91.0%
Episode: 8000, Recent Mean Reward: 7.25, Memory Usage: 92.6%
Episode: 8100, Recent Mean Reward: 7.33, Memory Usage: 92.4%
Episode: 8200, Recent Mean Reward: 6.75, Memory Usage: 93.2%
Episode: 8300, Recent Mean Reward: 7.25, Memory Usage: 92.3%
Episode: 8400, Recent Mean Reward: 7.25, Memory Usage: 81.2%
Episode: 8500, Recent Mean Reward: 6.85, Memory Usage: 85.5%
Episode: 8600, Recent Mean Reward: 7.26, Memory Usage: 88.5%
Episode: 8700, Recent Mean Reward: 7.25, Memory Usage: 93.2%
Episode: 8800, Recent Mean Reward: 7.25, Memory Usage: 94.6%
Episode: 8900, Recent Mean Reward: 7.25, Memory Usage: 93.1%
Episode: 9000, Recent Mean Reward: 7.25, Memory Usage: 93.8%
Episode: 9100, Recent Mean Reward: 7.25, Memory Usage: 93.2%
Episode: 9200, Recent Mean Reward: 7.03, Memory Usage: 93.3%
Episode: 9300, Recent Mean Reward: 7.06, Memory Usage: 85.7%
Episode: 9400, Recent Mean Reward: 6.61, Memory Usage: 93.5%
Episode: 9500, Recent Mean Reward: 7.13, Memory Usage: 93.5%
Episode: 9600, Recent Mean Reward: 6.55, Memory Usage: 95.3%
Episode: 9700, Recent Mean Reward: 7.25, Memory Usage: 94.1%
Episode: 9800, Recent Mean Reward: 7.25, Memory Usage: 94.8%
```

Figure 13: Performance of the Model for the Last 20 Episodes

Fig. 12 and Fig. 13 show the initial learning stage of the model, as in Fig. 12 the rewards recorded are very low and the values fluctuate a lot. Meanwhile, in Fig. 13 which shows the last 20 episodes of the model, there is a more consistent reward recorded and the values are consistently negative. This indicates that the agent was able to consistently find optimal solutions which finally led to the termination of the training process. This training took exactly 6 days for 10,000 episodes.

5. CONCLUSIONS

5.1 Summary

In this dissertation, numerous tests and experiments with reinforcement learning models were carried out to investigate the effectiveness of reinforcement learning models in adaptive control systems and how effectively they can be used to control the room temperature in an HVAC system. The numerous experiments carried out include the use of the Q-learning model to train a simple environment with the same initial temperature and a target temperature, then an experiment was carried out on the effect that occupancy fluctuation can have on the performance of the model, then double Q-learning model was implemented to see how reinforcement learning performs in extreme climate conditions, like in the winter and summer. There was a focus on the exploitation-exploration trade-off and their role in maintaining optimal temperature control. In this dissertation, it was evident that reinforcement learning models outperform PID controllers in terms of energy consumption, most especially when the environment is dynamic and constantly changing which is like a real-world environment.

5.2 Discussion

A very interesting observation in the results obtained from the experiments is how the SoftMax approach had always proven to be the best-performing exploration strategy in models that were evaluated during this project, but in the case of double Q-Learning, UCB performed better than both SoftMax and epsilon-greedy. Theoretically, this is because double Q-learning gives a less biased and more accurate estimated value, and UCB strives in a situation like this, on the other hand, Q-learning involves a lot of overestimations, and this is not so favourable for UCB, but SoftMax handles this very well.

5.3 Future Work

While this dissertation provides the basic framework for reinforcement learning-based adaptive control in HVAC systems, more work is needed and there are various avenues for future research. Few of the future work that can be done to improve on this project include:

- Testing in the real world: If the developed models can be implemented in a real-life environment where the conditions may be even more complex, it can validate the effectiveness of these models.
- Exploring more reinforcement learning models: In this project, deep Q-network was explored, and the models can further be improved with the use of more complex reinforcement learning models like deep learning.
- Smart Grids Integration: An investigation into how these models can be integrated with smart grids for further optimization of energy is a significant way to further improve on this project.

REFERENCES

[1] R. Sutton and A. Barto, ‘Reinforcement Learning: An Introduction’, p. 2, 2014.

[2] R. Sutton and A. Barto, ‘Reinforcement Learning: An Introduction’, p. 273, 2014.

[3] K. A. Sayed, A. Boodi, R. S. Broujeny, and K. Beddiar, ‘Reinforcement learning for HVAC control in intelligent buildings: A technical and conceptual review’, *J. Build. Eng.*, vol. 95, p. 110085, 2024, doi: https://doi.org/10.1016/j.jobe.2024.110085.

[4] S. Sierla, H. Ihasalo, and V. Vyatkin, ‘A Review of Reinforcement Learning Applications to Control of Heating, Ventilation and Air Conditioning Systems’, *Energies*, vol. 15, no. 10, 2022, doi: 10.3390/en15103526.

[5] P. Lissa, M. Schukat, and E. Barrett, ‘Transfer Learning Applied to Reinforcement Learning-Based HVAC Control’, *SN Comput. Sci.*, vol. 1, no. 3, p. 127, Apr. 2020, doi: 10.1007/s42979-020-00146-7.

[6] Poe, William A., and Saeid Mokhatab., ‘“Process Control.” Modeling, Control, and Optimization of Natural Gas Processing Plants.’, pp. 97–172, 2017.

[7] ‘Dynamics and Control’, Proportional Integral Derivative (PID). [Online]. Available: https://apmonitor.com/pdc/index.php/Main/ProportionalIntegralDerivative#:~:text=Derivative%20(D)%20control%3A%20This,because%20PI%20control%20is%20sufficient.

[8] R. P. Borase, D. K. Maghade, S. Y. Sondkar, and S. N. Pawar, ‘A review of PID control, tuning methods and applications’, *Springer*, pp. 821–824, Jul. 2020.

[9] C. Blasco, J. Monreal, I. Benítez, and A. Lluna, ‘Modelling and PID Control of HVAC System According to Energy Efficiency and Comfort Criteria’, in *Sustainability in Energy*

and Buildings, N. M'Sirdi, A. Namaane, R. J. Howlett, and L. C. Jain, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 365–374.

[10] J.-H. Urrea-Quintero, J. N. Fuhg, M. Marino, and A. Fau, 'PI/PID controller stabilizing sets of uncertain nonlinear systems: an efficient surrogate model-based approach', *Nonlinear Dyn.*, vol. 105, no. 1, pp. 277–299, Jul. 2021, doi: 10.1007/s11071-021-06431-1.